



دانشگاه صنعتی شریف

دانشکده مهندسی صنایع

گزارش take home امتحان پایان ترم درس برنامه ریزی تصادفی

نگارش

آرمین زینال زاده

استاد درس

جناب آقای دکتر رفیعی

شهریور 1404

قسمت اول) تولید سناریو های تصادفی و استفاده از روش کاهش سناریو

تولید سناریوهای اولیه

برای مدل سازی عدم قطعیت، ابتدا ۱۰۰۰ سناریوی اولیه تولید می شود. هر سناریو شامل مقادیری برای سه پارامتر تصادفی t_1, t_2, t_3 است. فرض بر این است که این پارامترها از یک توزیع نرمال با میانگین و انحراف معیار مشخص پیروی می کنند. این مجموعه بزرگ، فضای عدم قطعیت مسئله را به خوبی پوشش می دهد.

$$t_1 \sim N(\text{mean} = 2.5, \text{sd} = 1.5)$$

$$t_2 \sim N(\text{mean} = 3, \text{sd} = 2)$$

$$t_3 \sim N(\text{mean} = 20, \text{sd} = 7)$$

```
GENERATE INITIAL 1000 SCENARIO'S

land yields is assumed to follow a normal distribution

NUM_INITIAL_SCENARIOS = 1000

MEANS = {'t_1': 2.5, 't_2': 3, 't_3': 20}
STD_DEVS = {'t_1': 1.5, 't_2': 2.0, 't_3': 7.0}
coeffs = ['t_1', 't_2', 't_3']

initial_data = {}
for t in coeffs:
    initial_data[t] = np.random.normal(loc=MEANS[t], scale=STD_DEVS[t], size=NUM_INITIAL_SCENARIOS)

initial_scenarios_array = pd.DataFrame(initial_data).values

print(f"Generated {initial_scenarios_array.shape[0]} initial scenarios with {initial_scenarios_array.shape[1]} coeffs.\n")

[4] ✓ 0.0s
... Generated 1000 initial scenarios with 3 coeffs.
```

خروجی این کد یک ماتریس با 1000 سطر و 3 ستون میباشد.

```
initial_scenarios_array
[6] ✓ 0.0s
... array([[ 4.09708505,  4.6884044 , 18.91450279],
 [ 1.70208444,  2.62439579, 12.15043746],
 [ 2.57647907,  2.30848402, 19.73820699],
 ...,
 [ 2.09616592,  4.94460081, 23.31127547],
 [ 0.49094304,  0.59990454, 29.91943864],
 [ 3.2575345 ,  3.15101638, 17.12642394]])
```

الگوریتم کاهش سناریو (scenario_reduction)

هدف این الگوریتم کاهش ۱۰۰۰ سناریوی اولیه به ۱۰ سناریوی نهایی به همراه احتمالات بروز هر کدام است. مراحل این الگوریتم بصورت زیر می باشد:

- محاسبه فاصله: ابتدا فاصله اقلیدسی بین تمام زوج سناریوها محاسبه شده و یک ماتریس فاصله ایجاد می شود.
- انتخاب کاندیدای حذف: در هر مرحله، الگوریتم سناریویی را برای حذف انتخاب می کند که کمترین "فاصله وزنی" را به نزدیک ترین همسایه خود داشته باشد. فاصله با احتمال وقوع آن سناریو وزن دهی می شود تا از حذف سناریوهای مهم و محتمل جلوگیری شود.
- انتقال احتمال: پس از حذف یک سناریو، احتمال وقوع آن به نزدیک ترین سناریوی همسایه منتقل می شود. این کار تضمین می کند که مجموع احتمالات همواره یک باقی بماند و اهمیت آماری سناریوی حذف شده از بین نرود.
- تکرار: مراحل ۲ و ۳ آنقدر تکرار می شوند تا فقط ۱۰ سناریو در مجموعه باقی بماند.

بدین منظور یک تابع با نام (scenario_reduction) تعریف شد که دیکشنری داده های تولید شده را به عنوان ورودی میگیرد و الگوریتم کاهش سناریو را تا جایی ادامه میدهد که یا به target_size برسد و یا به مقدار epsilon.

تابع فوق فراخوانی شده و دیتاست اولیه به 10 سناریو کاهش یافت. برای نمایش مقادیر سناریو های انتخاب شده و احتمال آنها، خروجی های زیر بدست آمده است.

قابل ذکر است که مقادیر برخی از بازده ها منفی می باشد. این موضوع بطور شهودی نیز غیر منطقی نمی باشد از آنجائیکه امکان دارد مشکلاتی نظیر آفت، سبب کاهش بازده و حتی منفی شدن آن (زیان) شود. بنابراین مدل ارائه شده با همین مقادیر حل شده است.

APPLY SCENARIO REDUCTION FUNCTION TO GENERATED SCENARIOS

```

reduced_scenarios, reduced_probs = scenario_reduction(initial_scenarios_array, target_size=10)

print("Reduced Scenarios Shape:", reduced_scenarios.shape)
print("Reduced Probabilities:", reduced_probs)

[184]

... Reduced Scenarios Shape: (10, 3)
Reduced Probabilities: [0.003 0.158 0.109 0.001 0.001 0.496 0.225 0.001 0.001 0.005]

reduced_scenarios

[185]

... array([[ 3.40738221, -1.99895819, 32.49492946],
 [ 4.83128794,  3.95054638,  6.14392307],
 [ 4.12263418,  0.0918415 , 29.41611814],
 [ 0.43636268,  9.47950524, 29.03346194],
 [-0.29236374, -3.55512819,  8.08635326],
 [ 2.59497427, -3.60789161, 20.78525011],
 [ 3.14556892, -1.19392909, 35.94772189],
 [-0.62758228,  3.76085144, 14.10281658],
 [ 1.28512384,  5.87284087, 42.47299097],
 [ 2.74589963,  6.57000152,  6.58053332]])

for i in range(1, len(reduced_scenarios) + 1):
    print(f" scenario {i} : {reduced_scenarios[i-1]} with probability : {reduced_probs[i-1]}")

[186]

... scenario 1 : [ 3.40738221 -1.99895819 32.49492946] with probability : 0.003
scenario 2 : [4.83128794 3.95054638 6.14392307] with probability : 0.158
scenario 3 : [ 4.12263418  0.0918415 29.41611814] with probability : 0.10900000000000001
scenario 4 : [ 0.43636268  9.47950524 29.03346194] with probability : 0.001
scenario 5 : [-0.29236374 -3.55512819  8.08635326] with probability : 0.001
scenario 6 : [ 2.59497427 -3.60789161 20.78525011] with probability : 0.496
scenario 7 : [ 3.14556892 -1.19392909 35.94772189] with probability : 0.22500000000000003
scenario 8 : [-0.62758228  3.76085144 14.10281658] with probability : 0.001
scenario 9 : [ 1.28512384  5.87284087 42.47299097] with probability : 0.001
scenario 10 : [2.74589963 6.57000152 6.58053332] with probability : 0.005

```

فرمول‌بندی مدل بهینه‌سازی خطی (LP)

پس از به دست آوردن سناریوهای کاهش‌یافته، یک مدل LP جامع، فرمول‌بندی می‌گردد.

متغیرهای مساله بصورت زیر تعریف می‌شوند:

$x_1(i)$: متغیرهای تصمیم مرحله اول. میزان کشت محصول i برای $i \in [1,2,3]$

$x_2(i, s_2)$: میزان کشت محصول i در دوره دوم وقتی در دوره قبل سناریو s_2 محقق شده است.

$y_2(i, s_2)$: میزان کمبود محصول i در دوره دوم وقتی در دوره قبل سناریو s_2 محقق شده است.

$w_2(i, s_2)$: میزان مازاد محصول i در دوره دوم وقتی در دوره قبل سناریو s_2 محقق شده است.

$y_3(i, s_2, s_3)$: میزان کمبود محصول i در دوره سوم وقتی مسیر سناریو s_2 و s_3 طی شده است.

$w_3(i, s_2, s_3)$: میزان مازاد محصول i در دوره سوم وقتی مسیر سناریو s_2 و s_3 طی شده است.

مدلسازی

مدل نهایی بصورت زیر تعریف شده است. لازم به ذکر است که محدودیت منع کشت محصول سوم (چغندر قند) در دو سال پیاپی بدین صورت در نظر گرفته شده است که در برای هر بخش از زمین این محدودیت وجود دارد. یعنی در یک بخش مشخص از زمین دو سال پیاپی کشت محصول سوم انجام نخواهد شد. دلیل این فرض منطق ذکر شده در صورت سوال 6 صفحه 18 می باشد که بیان میکند برای جلوگیری از ضعیف شدن زمین کشت، در دو سال متوالی محصول چغندر قند قابل کشت نمی باشد. بنابراین در بخش هایی از زمین که در سال اول این محصول کشت نشده است، در سال دوم قابل کشت است.

مدل نهایی در پایتون و با استفاده از کتابخانه pulp بصورت زیر تعریف شده است:

```
x_1 = lpVariable.dicts('x_1', range(1, 4), lowBound=0, cat=lpInteger)
x_2 = lpVariable.dicts('x_2', ((i, s) for i in range(1, 4) for s in range(1, len(reduced_scenarios) + 1)), lowBound=0, cat=lpContinuous)
y_2 = lpVariable.dicts('y_2', ((i, s) for i in range(1, 4) for s in range(1, len(reduced_scenarios) + 1)), lowBound=0, cat=lpContinuous)
w_2 = lpVariable.dicts('w_2', ((i, s) for i in range(1, 5) for s in range(1, len(reduced_scenarios) + 1)), lowBound=0, cat=lpContinuous)

y_3 = lpVariable.dicts('y_3', ((i, s2, s3) for i in range(1, 4) for s2 in range(1, len(reduced_scenarios) + 1) for s3 in range(1, len(reduced_scenarios) + 1)), lowBound=0, cat='Continuous')
w_3 = lpVariable.dicts('w_3', ((i, s2, s3) for i in range(1, 5) for s2 in range(1, len(reduced_scenarios) + 1) for s3 in range(1, len(reduced_scenarios) + 1)), lowBound=0, cat='Continuous')

prob = lpProblem('LP', lpMinimize)
cost_1 = 150 * x_1[1] + 230 * x_1[2] + 260 * x_1[3]
cost_2 = lpSum((reduced_probs[s2-1] * (150 * x_2[1, s2] + 230 * x_2[2, s2] + 260 * x_2[3, s2] + 238 * y_2[1, s2] - 170 * w_2[1, s2]
+ 210 * y_2[2, s2] - 150 * w_2[2, s2] - 36 * w_2[3, s2] - 10 * w_2[4, s2]) for s2 in range(1, len(reduced_scenarios)+1))

cost_3 = lpSum(((reduced_probs[s2-1] * reduced_probs[s3-1])) * (238 * y_3[1, s2, s3] - 170 * w_3[1, s2, s3] + 210 * y_3[2, s2, s3]
- 150 * w_3[2, s2, s3] - 36 * w_3[3, s2, s3] - 10 * w_3[4, s2, s3])
for s2 in range(1, len(reduced_scenarios)+1) for s3 in range(1, len(reduced_scenarios)+1))

prob += cost_1 + cost_2 + cost_3

prob += x_1[1] + x_1[2] + x_1[3] <= 500 # first stage

for s in range(1, len(reduced_scenarios)+1):
    prob += x_1[3] + x_2[3, s] <= 500

# second stage
for s in range(1, len(reduced_scenarios)+1):
    t_1 = reduced_scenarios[s-1][0]
    t_2 = reduced_scenarios[s-1][1]
    t_3 = reduced_scenarios[s-1][2]

    prob += x_2[1, s] + x_2[2, s] + x_2[3, s] <= 500

    prob += y_2[1, s] - w_2[1, s] >= 200 - t_1 * x_1[1]
    prob += y_2[2, s] - w_2[2, s] >= 240 - t_2 * x_1[2]
    prob += w_2[3, s] + w_2[4, s] <= t_3 * x_1[3]
    prob += w_2[3, s] <= 6000

# third stage
for s2 in range(1, len(reduced_scenarios)+1):
    for s3 in range(1, len(reduced_scenarios)+1):
        t_1 = reduced_scenarios[s3-1][0]
        t_2 = reduced_scenarios[s3-1][1]
        t_3 = reduced_scenarios[s3-1][2]

        prob += y_3[1, s2, s3] - w_3[1, s2, s3] >= 200 - t_1 * x_2[1, s2]
        prob += y_3[2, s2, s3] - w_3[2, s2, s3] >= 240 - t_2 * x_2[2, s2]
        prob += w_3[3, s2, s3] + w_3[4, s2, s3] <= t_3 * x_2[3, s2]
        prob += w_3[3, s2, s3] <= 6000
```

حل مدل

پس از مدلسازی انجام شده، مدل حل شده و نتایج زیر بدست آمده است :

مقادیر بهینه کشت در مرحله اول و مقادیر بهینه کشت، خرید و فروش در مرحله دوم به ازای هر سناریو

(با توجه به تعداد بالای سناریو ها در خروجی نمایش داده شده فقط تعدادی از سناریو پوشش داده شده

است)

```
prob.solve()
print(f"objective value is {value(prob.objective)}\n")
print(f"first stage decisions is X : {[x_1[s].varValue for s in range(1, 4)]}\n")
for s in range(1, len(reduced_scenarios) + 1):
    print(f"second stage decision : x_2 for scenario {s}: {[x_2[i, s].varValue for i in range(1, 4)]}")
    print(f"second stage decision : y_2 for scenario {s}: {[y_2[i, s].varValue for i in range(1, 3)]}")
    print(f"second stage decision : w_2 for scenario {s}: {[w_2[i, s].varValue for i in range(1, 5)]}\n")
```

[43] ✓ 0.0s

... objective value is -242109.13061740002

first stage decisions is X : [500.0, 0.0, 0.0]

second stage decision : x_2 for scenario 1: [500.0, 0.0, 0.0]

second stage decision : y_2 for scenario 1: [0.0, 240.0]

second stage decision : w_2 for scenario 1: [527.95367, 0.0, 0.0, 0.0]

second stage decision : x_2 for scenario 2: [500.0, 0.0, 0.0]

second stage decision : y_2 for scenario 2: [0.0, 240.0]

second stage decision : w_2 for scenario 2: [1957.2561, 0.0, 0.0, 0.0]

second stage decision : x_2 for scenario 3: [500.0, 0.0, 0.0]

second stage decision : y_2 for scenario 3: [0.0, 240.0]

second stage decision : w_2 for scenario 3: [3459.7646, 0.0, 0.0, 0.0]

second stage decision : x_2 for scenario 4: [500.0, 0.0, 0.0]

second stage decision : y_2 for scenario 4: [0.0, 240.0]

second stage decision : w_2 for scenario 4: [1174.1715, 0.0, 0.0, 0.0]

second stage decision : x_2 for scenario 5: [500.0, 0.0, 0.0]

second stage decision : y_2 for scenario 5: [814.17372, 240.0]

second stage decision : w_2 for scenario 5: [0.0, 0.0, 0.0, 0.0]

second stage decision : x_2 for scenario 6: [500.0, 0.0, 0.0]

...

second stage decision : x_2 for scenario 10: [500.0, 0.0, 0.0]

second stage decision : y_2 for scenario 10: [490.53238, 240.0]

second stage decision : w_2 for scenario 10: [0.0, 0.0, 0.0, 0.0]

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

و مقادیر بهینه خرید و فروش محصولات در مرحله سوم به ازای هر مسیر طی شده :

```
for s2 in range(1, len(reduced_scenarios) + 1):
    for s3 in range(1, len(reduced_scenarios * reduced_scenarios)+1):
        print(f"third stage decision : y_3 for path {s2, s3}: {[y_3[i, s2, s3].varValue for i in range(1, 3)]}")
        print(f"third stage decision : w_3 for path {s2, s3}: {[w_3[i, s2, s3].varValue for i in range(1, 5)]}\n")

[52] ✓ 0.0s

...
third stage decision : y_3 for path (1, 1): [0.0, 240.0]
third stage decision : w_3 for path (1, 1): [527.95367, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 2): [0.0, 240.0]
third stage decision : w_3 for path (1, 2): [1957.2561, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 3): [0.0, 240.0]
third stage decision : w_3 for path (1, 3): [3459.7646, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 4): [0.0, 240.0]
third stage decision : w_3 for path (1, 4): [1174.1715, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 5): [814.17372, 240.0]
third stage decision : w_3 for path (1, 5): [0.0, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 6): [0.0, 240.0]
third stage decision : w_3 for path (1, 6): [921.31194, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 7): [1265.4939, 240.0]
third stage decision : w_3 for path (1, 7): [0.0, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 8): [0.0, 240.0]
third stage decision : w_3 for path (1, 8): [2206.1279, 0.0, 0.0, 0.0]

third stage decision : y_3 for path (1, 9): [0.0, 240.0]
...

third stage decision : y_3 for path (10, 10): [490.53238, 240.0]
third stage decision : w_3 for path (10, 10): [0.0, 0.0, 0.0, 0.0]

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

با توجه به اینکه در مرحله دوم 10 سناریو ممکن وجود دارد، در مرحله سوم نیز به ازای هر سناریو محقق شده در مرحله دوم 10 سناریو وجود دارد. این به این معنی است که تعداد برگ های نهایی درخت سناریو برابر 100 میباشد. در شکل بالا خروجی نهایی به ازای تعدادی از مسیر های طی شده برای متغیر های تصمیم مرحله سوم ارائه شده است.

بخش دوم) توزیع گسسته و الگوریتم تو در تو

برای انجام این بخش، ابتدا به شرح نحوه تولید سناریو ها و تحقق فرض گسسته بودن خواهیم پرداخت. سپس نحوه تعریف مسائل، تعریف درخت سناریو و نگاشت های آن خواهیم پرداخت و در نهایت به شرح توابع تعریف شده و منطق اصلی کد میردازیم.

سناریو ها)

از آنجائیکه فرض شده است بازده زمین دارای توزیع گسسته می باشد، در این قسمت فرض کرده ایم این بازده ها از توزیع گسسته یکنواخت پیروی میکنند. بدین منظور ابتدا لیستی از بازده های ممکن برای هر محصول ایجاد شد.

```
ASSUMED YIELD'S FOLLOWS DISCRETE UNIFORM

possible_t_1 = np.arange(2, 3.1, 0.1)
possible_t_2 = np.arange(2.4, 3.7, 0.1)
possible_t_3 = np.arange(16, 25, 1)

[43] ✓ 0.0s

print(f"{possible_t_1}\n{possible_t_2}\n{possible_t_3}")

[44] ✓ 0.0s

... [2.  2.1 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9 3. ]
     [2.4 2.5 2.6 2.7 2.8 2.9 3.  3.1 3.2 3.3 3.4 3.5 3.6 3.7]
     [16 17 18 19 20 21 22 23 24]
```

به عنوان مثال بازده محصول اول میتواند از بازه 2 الی 3 با گام های 0.1 باشد.

سپس مشابه قسمت قبل برای تعریف سناریو، از یک دیکشنری استفاده شد که هر کلید نشان دهنده سناریو و مقدار آن نیز یک دیکشنری حاوی مقادیر بازده می باشد. مقادیر بازده برای هر سناریو بصورت تصادفی (با استفاده از np.random.choice) انتخاب شده اند.

کد قسمت مربوطه همراه با خروجی آن در شکل زیر ارائه شده است.


```

num_scenarios = 3
scenario_stage2 = {s: {"t_1": np.random.choice(possible_t_1),
                      "t_2": np.random.choice(possible_t_2),
                      "t_3": np.random.choice(possible_t_3)} for s in range(1, num_scenarios + 1)}

scenario_stage3 = {s: {"t_1": np.random.choice(possible_t_1),
                      "t_2": np.random.choice(possible_t_2),
                      "t_3": np.random.choice(possible_t_3)} for s in range(1, num_scenarios * num_scenarios + 1)}

```

[45] ✓ 0.0s

scenario_stage2

[46] ✓ 0.0s

```

... {1: {'t_1': 2.8000000000000007, 't_2': 3.6000000000000001, 't_3': 24},
     2: {'t_1': 2.2, 't_2': 3.1000000000000005, 't_3': 16},
     3: {'t_1': 2.9000000000000001, 't_2': 2.6, 't_3': 16}}

```

scenario_stage3

[47] ✓ 0.0s

```

... {1: {'t_1': 3.0000000000000001, 't_2': 3.7000000000000001, 't_3': 19},
     2: {'t_1': 2.3000000000000003, 't_2': 3.1000000000000005, 't_3': 18},
     3: {'t_1': 2.1, 't_2': 2.5, 't_3': 23},
     4: {'t_1': 2.8000000000000007, 't_2': 2.9000000000000004, 't_3': 21},
     5: {'t_1': 2.4000000000000004, 't_2': 2.7, 't_3': 19},
     6: {'t_1': 2.1, 't_2': 2.7, 't_3': 18},
     7: {'t_1': 2.4000000000000004, 't_2': 2.5, 't_3': 20},
     8: {'t_1': 3.0000000000000001, 't_2': 3.5000000000000001, 't_3': 16},
     9: {'t_1': 2.2, 't_2': 3.6000000000000001, 't_3': 21}}

```

Generate + Code + Markdown

همانطور که ملاحظه میشود، تعداد سناریو های ممکن برای مرحله دوم 3 و برای مرحله سوم 9 سناریو در نظر گرفته شده است که برای هر مرحله یک دیکشنری حاوی سناریو ها و مقادیر آن تعریف شده است.

تعریف مسائل و نگاشت)

مساله مذکور سه مرحله دارد که مرحله اول شامل یک مساله قطعی، مرحله دوم شامل سه مساله و مرحله سوم شامل 9 مساله می باشد. برای اجرای الگوریتم تو در تو ضروری است که هر یک از این مسائل را با نام $NLDS(t, k)$ تعریف کرده و سناریو های فرزند و والد را شناسایی بکنیم.

برای این منظور، هر یک از این مسائل را با یک node مشخص میکنیم. هر یک tuple می باشد که به فرم (t, k) تعریف میشود. به عنوان مثال مساله $NLDS(t=2, k=3)$ با گره (3 و 2) تعریف میشود.

برای تعریف و دسته بندی این گره ها، یک دیکشنری تعریف میکنیم که value های آن حاوی node ها هستند.

FOR EACH $NLDS(t, k)$ A TUPLE (t, k) IS DEFINED

```

nodes= {
    1 : (1, 1),
    2 : (2, 1), 3 : (2, 2), 4 : (2, 3),
    5 : (3, 1), 6 : (3, 2), 7 : (3, 3),
    8 : (3, 4), 9 : (3, 5), 10 : (3, 6),
    11 : (3, 7), 12 : (3, 8), 13 : (3, 9)
}

```

[48] ✓ 0.0s

برای تعریف روابط بین گره ها (رابطه والدی یا ولدی) 2 دیکشنری دیگر تعریف میکنیم. این دیکشنری ها از اهمیت ویژه ای در پیمایش گره ها در مسیر رفت و برگشت الگوریتم تو در تو برخوردار هستند.

```
DEFINE A MAP OF SCENARIOS FOR IDENTIFYING PARENT AND CHILD NODES

child_to_parent_map = {
    (2, 1): (1, 1), (2, 2): (1, 1), (2, 3): (1, 1),
    (3, 1): (2, 1), (3, 2): (2, 1), (3, 3): (2, 1),
    (3, 4): (2, 2), (3, 5): (2, 2), (3, 6): (2, 2),
    (3, 7): (2, 3), (3, 8): (2, 3), (3, 9): (2, 3),
}

parent_to_children_map = {
    (1, 1): [(2, 1), (2, 2), (2, 3)],
    (2, 1): [(3, 1), (3, 2), (3, 3)],
    (2, 2): [(3, 4), (3, 5), (3, 6)],
    (2, 3): [(3, 7), (3, 8), (3, 9)],
}
```

ایده اصلی پیاده سازی الگوریتم تو در تو تعریف دیکشنری های parent_to_children_map و child_to_parent_map می باشد. در ادامه به تعریف توابع و منطق اصلی کد خواهیم پرداخت.

توابع کلیدی و منطق پیاده سازی (

برای جلوگیری از طولانی شدن گزارش این توابع بصورت خلاصه معرفی می شوند.

تابع get_coeffs :

این تابع یک مساله خطی و لیست متغیرهای والد و لیست متغیرهای مساله فعلی را دریافت کرده، مقادیر سمت راست آن و ضرایب متغیرهای parent آن مساله را استخراج کرده و برمیگرداند. یکی از چالشهای اساسی پیاده سازی این الگوریتم، استخراج ضرایب h و T بود. با توجه به اینکه مسائل پویا هستند (به این معنی که برش های جدید به آنها اضافه می شود) ضروری است که در هر تکرار الگوریتم این ضرایب استخراج شوند. (قابل ذکر است که مقادیر موجود در این ماتریس ها تغییری نمیکند چرا که برش های اضافه شده فقط بر اساس متغیرهای همان مرحله می باشد اما طول این ماتریس ها تغییر می یابد). بنابراین باید در هر تکرار این ضرایب استخراج شوند. بدین منظور این تابع تعریف شده است.

مدل های نمادین (symbolic_models) :

از طرفی در مرحله رفت الگوریتم، مسائلی حل میشوند که متغیرهای مرحله قبل بصورت عددی وارد شده اند. این امر موجب می شود استخراج ماتریس های h و T غیرممکن باشد چرا که با تعریف یک مساله با استفاده از پالپ و جایگذاری `x_parent[i].varValue` در محدودیت ها، کتابخانه پالپ این ترم را به عنوان یک عدد میبیند بنابراین نمیتوان ضریب این عدد را استخراج نمود.

برای حل این مشکل مدل های سمبولیک تعریف شده اند. در این مدلها متغیرهای والد موجود در محدودیت ها به شکل متغیر تعریف شده اند (و نه جواب بهینه گره والد). بنابراین برای استخراج ماتریس های ضرایب، از این مسائل استفاده می شود.

برای ذخیره کردن این مدلها از یک دیکشنری به نام `symbolic_models` استفاده شده است که کلید آن `node` و مقدار آن مساله سمبولیک میباشد.

```
symbolic_models
[54] ✓ 0.0s
...
{1,
  1): Symbolic_s1_n1:
  MINIMIZE
  1*theta_s1_n1 + 150*x_s1_n1_1 + 230*x_s1_n1_2 + 260*x_s1_n1_3 + 0
  SUBJECT TO
  _C1: x_s1_n1_1 + x_s1_n1_2 + x_s1_n1_3 <= 500

  VARIABLES
  -1000000000 <= theta_s1_n1 Continuous
  x_s1_n1_1 Continuous
  x_s1_n1_2 Continuous
  x_s1_n1_3 Continuous,
  2,
  1): Symbolic_s2_n1:
  MINIMIZE
  1*theta_s2_n1 + -170*w_s2_n1_1 + -150*w_s2_n1_2 + -36*w_s2_n1_3 + -10*w_s2_n1_4 + 150*x_s2_n1_1 + 230*x_s2_n1_2 + 260*x_s2_n1_3 + 238*y_s2_n1_1 + 210*y_s2_n1_2 + 0
  SUBJECT TO
  _C1: x_s2_n1_1 + x_s2_n1_2 + x_s2_n1_3 <= 500

  _C2: - w_s2_n1_1 + 2.8 x_s1_n1_1 + y_s2_n1_1 >= 200

  _C3: - w_s2_n1_2 + 3.6 x_s1_n1_2 + y_s2_n1_2 >= 240

  _C4: - w_s2_n1_3 - w_s2_n1_4 + 24 x_s1_n1_3 >= 0

  ...
  x_s2_n3_1 Continuous
  x_s2_n3_2 Continuous
  x_s2_n3_3 Continuous
  y_s3_n9_1 Continuous
  y_s3_n9_2 Continuous}
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

تابع (build_w_prime)

این تابع یک مساله را میگیرد و بر اساس آن مساله بررسی شدنی بودن را تولید میکند.

تابع (solve_subproblem)

این تابع مسئول حل زیرمسئله‌ی بهینه‌سازی برای هر گره دلخواه در درخت است. ورودی‌های این تابع عبارتند از:

- Node : گره‌ای که باید حل شود
 - Parent_solution : جواب بهینه گره والد
 - Existing_cuts : برش‌هایی که در تکرارهای قبلی به این مساله اضافه شده‌اند.
- این تابع در داخل خود، یک مدل بهینه‌سازی کاملاً جدید و محلی با استفاده از کتابخانه PuLP می‌سازد. این مدل بهینه‌سازی مقادیر بهینه گره والد را دربر دارد (برخلاف مدل سمبولیک) سپس، با استفاده از parent_solution، محدودیت‌هایی که مراحل را به هم پیوند می‌دهند، ایجاد می‌کند و در نهایت، برش‌های (existing_cuts) دریافتی را به مدل اضافه می‌کند و حل می‌کند.
- این تابع همیشه یک تاپل دوتایی برمی‌گرداند: (status, result).

1. Status : وضعیت شدنی بودن یا نبودن مساله را نشان می‌دهد. یک رشته متنی است.
 2. Results : دیکشنری نتایج که به نتیجه status بستگی دارد و شامل دو کلید solution و duals
- اگر مسئله نشدنی باشد، الگوریتم یک مسئله (feasibility-check problem) درست کرده و حل می‌کند. با استفاده از تابع (build_w_prime). خروجی این مسئله کمکی، مجموعه‌ای از دوگان‌هاست که در کلید 'sigma' ذخیره می‌شوند. این دوگان‌ها دقیقاً همان چیزی هستند که تابع generate_feasibility_cut برای ساختن یک برش و جلوگیری از این حالت نشدنی در تکرار بعدی نیاز دارد.
- به طور خلاصه، solve_subproblem نه تنها مسئله هر گره را حل می‌کند، بلکه تمام داده‌های اولیه (جواب‌های primal و dual) که برای فاز بعدی الگوریتم (چه حرکت به جلو و چه بازگشت به عقب) لازم است را آماده می‌کند.

تابع generate_feasibility_cut (

در صورتی که وضعیت یک مساله "infeasible" باشد، این تابع فراخوانی خواهد شد. این تابع با استفاده از خروجی های تابع solve_subproblem و با فراخوانی تابع get_coeffs تمام ماتریس هایی که برای تولید برش شدنی بودن نیاز دارد را بدست آورده و یک برش شدنی بودن تولید کرده و آن را به گره parent اضافه میکند.

تابع generate_optimality_cut (

مشابه تابع generate_feasibility_cut، یک برش بهینگی تولید کرده و به مساله parent اضافه میکند. این تابع موتور اصلی قسمت backward الگوریتم میباشد. ورودی های کلیدی

parent_node: گره ای که می خواهیم برای آن برش بسازیم.

children_nodes: لیست گره های فرزند که بلافاصله بعد از گره پدر قرار دارند.

solutions و duals: این دو دیکشنری، حاوی تمام اطلاعات به دست آمده از مسیر رفت هستند که توسط تابع solve_subproblem ساخته شده اند.

تابع یک دیکشنری به نام cut_info برمی گرداند. نکته بسیار مهم این است که این دیکشنری، خود عبارت برش به صورت یک آبجکت PuLP را شامل نمی شود؛ بلکه داده های خام سازنده برش را برمی گرداند:

'type': نوع برش که در اینجا 'optimality' است.

'E_coeffs': آرایه ای از ضرایب E

'e_rhs': مقدار عددی e (مقدار سمت راست برش).

در صورتیکه نیازی به برش نبود تابع مقدار None برمی گرداند.

ساختار اصلی الگوریتم و حلقه تکرار

کل فرآیند در یک حلقه while مدیریت می‌شود که تا زمان بهینه شدن ($is_optimal = True$) ادامه دارد. این حلقه دو فاز اصلی دارد که توسط متغیر direction کنترل می‌شوند: مسیر رفت ("FORE") و مسیر برگشت ("BACK").

۱. مقداردهی اولیه (Initialization)

قبل از شروع حلقه، متغیرهای اصلی مقداردهی می‌شوند:

solutions, duals, cuts: سه دیکشنری خالی برای ذخیره نتایج حل، مقادیر دوگان و برش‌های تولید شده برای هر گره.

direction = "FORE": الگوریتم همیشه با مسیر رفت شروع می‌شود.

current_stage = 1, current_scenario_index = 1: نقطه شروع الگوریتم، اولین گره در مرحله اول است

۲. مسیر رفت (Forward Pass)

حلقه از گره (1, 1) شروع می‌شود.

تابع solve_subproblem برای گره فعلی فراخوانی می‌شود.

اگر جواب امکان‌پذیر (feasible) بود:

نتایج (solution) و مقادیر دوگان (duals) در دیکشنری‌ها ذخیره می‌شوند.

الگوریتم به گره بعدی در درخت سناریو حرکت می‌کند

اگر جواب نشدنی (infeasible) بود:

یک برش امکان‌سنجی (Feasibility Cut) با کمک generate_feasibility_cut تولید می‌شود.

این برش به گره parent اضافه می‌شود.

الگوریتم یک مرحله به عقب بازمی‌گردد تا گره parent را دوباره با این برش جدید حل کند و از ایجاد مجدد حالت نشدنی جلوگیری کند.

پایان فاز: این مسیر تا زمانی که آخرین گره در آخرین مرحله با موفقیت حل شود، ادامه می‌یابد. پس از آن، متغیر direction به "BACK" تغییر می‌کند تا فاز بعدی آغاز شود.

۳. مسیر برگشت (Backward Pass)

الگوریتم به مرحله یکی مانده به آخر بازمی‌گردد.

برای هر "گره والد" در این مرحله، تابع generate_optimality_cut فراخوانی می‌شود. این تابع با

استفاده از مقادیر دوگان گره‌های ولد، یک برش بهینگی (Optimality Cut) می‌سازد.

اگر برش جدیدی تولید شود، در دیکشنری cuts ذخیره می‌شود تا در تکرار بعدی مسیر رفت، به مدل آن گره اضافه شود.

این فرآیند تا رسیدن به مرحله اول ادامه می‌یابد.

پایان فاز: پس از بررسی مرحله ۱، این فاز به پایان می‌رسد و الگوریتم برای شروع یک تکرار جدید تصمیم‌گیری می‌کند.

شرط توقف : تا زمانی که هیچ برشی به مساله مرحله اول اضافه نشود حلقه تکرار می‌شود.

تصویر خروجی نهایی در دو صفحه بعد ارائه شده است.

```

1 |
2 | ===== STARTING ITERATION 1 =====
3 | --- FORWARD PASS ---
4 | Solving Node: (1, 1)
5 | | | | | | | | | | Feasible.
6 |
7 | >>> Stage 1 Results for Iteration 1:
8 | | Decision Variables: [ x1: 0.00, x2: 0.00, x3: 0.00 ]
9 | | Objective (Cost + Theta): -100000000.00
10 |
11 | Solving Node: (2, 1)
12 | | | | | | | | | | Feasible.
13 |
14 | Solving Node: (2, 2)
15 | | | | | | | | | | Feasible.
16 |
17 | Solving Node: (2, 3)
18 | | | | | | | | | | Feasible.
19 |
20 | Solving Node: (3, 1)
21 | | | | | | | | | | Feasible.
22 |
23 | Solving Node: (3, 2)
24 | | | | | | | | | | Feasible.
25 |
26 | Solving Node: (3, 3)
27 | | | | | | | | | | Feasible.
28 |
29 | Solving Node: (3, 4)
30 | | | | | | | | | | Feasible.
31 |
32 | Solving Node: (3, 5)
33 | | | | | | | | | | Feasible.
34 |
35 | Solving Node: (3, 6)
36 | | | | | | | | | | Feasible.
37 |
38 | Solving Node: (3, 7)
39 | | | | | | | | | | Feasible.
40 |
41 | Solving Node: (3, 8)
42 | | | | | | | | | | Feasible.
43 |
44 | Solving Node: (3, 9)
45 | | | | | | | | | | Feasible.
46 |
47 |
48 | --- BACKWARD PASS ---
49 | Generating optimality cuts for Stage 2
50 | -> Optimality gap for (2, 1). Generating cut.
51 | Generated Optimality Cut:  $\theta_{s2\_n1} + 587.07 * x_{s2\_n1\_1} + 651.00 * x_{s2\_n1\_2} + 720.00 * x_{s2\_n1\_3} \geq 98000.00$  for (2, 1)
52 |
53 | -> Optimality gap for (2, 2). Generating cut.
54 | Generated Optimality Cut:  $\theta_{s2\_n2} + 579.13 * x_{s2\_n2\_1} + 581.00 * x_{s2\_n2\_2} + 696.00 * x_{s2\_n2\_3} \geq 98000.00$  for (2, 2)
55 |
56 | -> Optimality gap for (2, 3). Generating cut.
57 | Generated Optimality Cut:  $\theta_{s2\_n3} + 602.93 * x_{s2\_n3\_1} + 672.00 * x_{s2\_n3\_2} + 684.00 * x_{s2\_n3\_3} \geq 98000.00$  for (2, 3)
58 |
59 | Generating optimality cuts for Stage 1
60 | -> Optimality gap for (1, 1). Generating cut.
61 | Generated Optimality Cut:  $\theta_{s1\_n1} + 626.73 * x_{s1\_n1\_1} + 651.00 * x_{s1\_n1\_2} + 672.00 * x_{s1\_n1\_3} \geq 98000.00$  for (1, 1)
62 |
63 |

```



```

64 ===== STARTING ITERATION 2 =====
65 --- FORWARD PASS ---
66 Solving Node: (1, 1)
67 | | | | | | | | | | Feasible.
68
69 >>> Stage 1 Results for Iteration 2:
70 | | | Decision Variables: [ x1: 500.00, x2: 0.00, x3: 0.00 ]
71 | | | Objective (Cost + Theta): -140366.67
72
73 Solving Node: (2, 1)
74 | | | | | | | | | | Feasible.
75
76 Solving Node: (2, 2)
77 | | | | | | | | | | Feasible.
78
79 Solving Node: (2, 3)
80 | | | | | | | | | | Feasible.
81
82 Solving Node: (3, 1)
83 | | | | | | | | | | Feasible.
84
85 Solving Node: (3, 2)
86 | | | | | | | | | | Feasible.
87
88 Solving Node: (3, 3)
89 | | | | | | | | | | Feasible.
90
91 Solving Node: (3, 4)
92 | | | | | | | | | | Feasible.
93
94 Solving Node: (3, 5)
95 | | | | | | | | | | Feasible.
96
97 Solving Node: (3, 6)
98 | | | | | | | | | | Feasible.
99
100 Solving Node: (3, 7)
101 | | | | | | | | | | Feasible.
102
103 Solving Node: (3, 8)
104 | | | | | | | | | | Feasible.
105
106 Solving Node: (3, 9)
107 | | | | | | | | | | Feasible.
108
109
110 --- BACKWARD PASS ---
111 Generating optimality cuts for Stage 2
112 -> Optimality gap for (2, 1). Generating cut.
113 Generated Optimality Cut:  $\theta_{s2\_n1} + 587.07 * x_{s2\_n1\_1} + 651.00 * x_{s2\_n1\_2} + 200.00 * x_{s2\_n1\_3} \geq -58000.00$  for (2, 1)
114
115 -> Optimality gap for (2, 2). Generating cut.
116 Generated Optimality Cut:  $\theta_{s2\_n2} + 579.13 * x_{s2\_n2\_1} + 581.00 * x_{s2\_n2\_2} + 193.33 * x_{s2\_n2\_3} \geq -58000.00$  for (2, 2)
117
118 -> Optimality gap for (2, 3). Generating cut.
119 Generated Optimality Cut:  $\theta_{s2\_n3} + 430.67 * x_{s2\_n3\_1} + 672.00 * x_{s2\_n3\_2} + 684.00 * x_{s2\_n3\_3} \geq 84400.00$  for (2, 3)
120
121 Generating optimality cuts for Stage 1
122
123 Convergence reached: No new cut added to Stage 1.
124
125 --- FINAL OPTIMAL SOLUTION ---
126 First stage decision (x_1): [500.0, 0.0, 0.0]
127 Total expected cost: -140366.67
128

```

بخش سوم (مقایسه نتایج

در قسمت اول، تصمیم بهینه مرحله اول بصورت $(500, 0, 0)$ می باشد که به معنی کشت گندم است و در مرحله دوم نیز تصمیم کشت همیشه (به ازای تمامی سناریو ها) کشت گندم می باشد. حل بدست آمده از روش تو در تو نیز به همین صورت می باشد. در مرحله اول تصمیم کشت همیشه کشت گندم می باشد اما در مرحله دوم به ازای سناریو سوم این تصمیم از کشت گندم به کشت چغندر قند تغییر میکند :

```
--- FINAL OPTIMAL SOLUTION ---  
  
First stage decision (x_1): [500.0, 0.0, 0.0]  
Total expected cost: -140366.67  
  
--- Second Stage Recourse Decisions ---  
For Scenario 1 (Node (2, 1)):  
| Optimal second-stage decision (x_2): [ x1: 0.00, x2: 0.00, x3: 500.00 ]  
For Scenario 2 (Node (2, 2)):  
| Optimal second-stage decision (x_2): [ x1: 0.00, x2: 0.00, x3: 500.00 ]  
For Scenario 3 (Node (2, 3)):  
| Optimal second-stage decision (x_2): [ x1: 500.00, x2: 0.00, x3: 0.00 ]
```

اما مقدار تابع هدف در بخش اول کمتر از بخش دوم می باشد. دلیل این موضوع را میتوان در منفی بودن بازده برخی محصولات در برخی سناریو ها در بخش اول دانست. این درحالیست که در بخش تو در تو، بازده ها همیشه مثبت و داخل یک بازه تعریف شده اند.